

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Sugith Marini B
Register Number	3122247001065
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20.09.2026

1 Objective

The objective of this experiment is to implement a Single-Layer Perceptron Learning Algorithm (PLA) and a Multilayer Perceptron (MLP) from the ground up, and to compare their ability to recognise handwritten characters. The MLP is tuned over several combinations of activation function, optimizer, learning rate, batch size and network depth, and the two models are then judged side by side on the same held-out test set.

2 Problem Statement

A handwritten character, given as a scanned image, has to be classified into one of 62 categories (digits 0–9, uppercase A–Z, lowercase a–z).

- **Input:** a grayscale image of a single handwritten character.
- **Output:** the predicted character class, one of 62 possible labels.
- **Goal:** measure how much a single linear decision boundary (PLA) loses against a non-linear, multi-layer model (MLP) on the same 62-way classification task, and identify which MLP hyperparameters matter most.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (sourced from the NIST handwritten character collection)
Number of Samples	3,410 images
Number of Features	1,024 (32×32 grayscale pixels, flattened)
Number of Classes	62: digits 0–9, uppercase A–Z, lowercase a–z
Missing Values	None. Every one of the 62 classes has exactly 55 images, so the dataset is perfectly balanced
Train-Test Split	80% train (2,728 images) / 20% test (682 images), stratified by class, <code>random_state=42</code>

4 Exploratory Data Analysis

Dataset overview and statistical summary

Each original image is 1200×900 pixels in RGB. Every character folder in `english.csv` maps to exactly 55 samples, giving a perfectly balanced 62-class problem with no class-imbalance correction required.

Missing value analysis

The dataset is complete: the label file lists exactly 3,410 valid image paths and no row has a missing image or label.

Class distribution

Because every class holds exactly 55 images (minimum count = maximum count = 55), a bar chart of class frequency is a flat line and carries no information; this was confirmed programmatically instead of plotted. The practical consequence is that macro averaging is the correct choice for precision, recall and F1, since no class is under- or over-represented.

Sample images

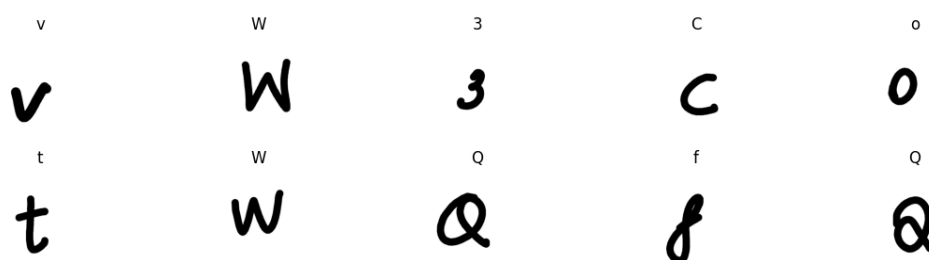


Figure 1: Ten randomly sampled characters with their ground-truth labels.

Inference: The sample confirms that different people’s handwriting styles vary considerably in stroke thickness and slant, and that some classes are visually close to one another (for example the uppercase Q and W on the strip differ mainly in a small tail or a sharper point). Every image is a single character on a plain white background, with no surrounding noise. This visual overlap between certain letters is one of the reasons a purely linear model struggles on this dataset.

Pixel value distribution

The raw pixel values, after scaling to $[0, 1]$, have a mean of 0.942, since each image is mostly blank paper. This single statistic became the most consequential preprocessing decision in the whole experiment and is discussed in Section 6.

5 Data Preprocessing

- **Resizing:** every image was converted to grayscale and resized from 1200×900 to 32×32 , giving a fixed length feature vector of 1,024 pixels per image; this keeps the PLA and MLP fast enough to tune within a lab session.
- **Flattening:** each 32×32 image was flattened into a single row vector of 1,024 values.
- **Normalization:** pixel intensities were scaled from the 0–255 range to $[0, 1]$ by dividing by 255.

- **Pixel inversion:** the scanned characters are dark ink on white paper, so the scaled images had a mean pixel value of 0.942, meaning that over 94 % of every image is background. Feeding this directly into the MLP gave a test accuracy of only 3.5 %, effectively random guessing over 62 classes, because the ink (the only informative part of the image) was numerically drowned out by the background. Inverting the pixels with $X \leftarrow 1 - X$ makes the ink close to 1 and the background close to 0 (mean pixel value after inversion: 0.058), which raised the same untuned MLP to roughly 48 % accuracy in preliminary testing. This single step mattered more than any hyperparameter tested later.
- **Label encoding:** the 62 character labels were mapped to integers 0–61 using `LabelEncoder`.
- **Train-test split:** an 80/20 stratified split (`random_state=42`) was used so that every class is represented proportionally in both the training set (2,728 images) and the test set (682 images).

6 Machine Learning Algorithm

Single-Layer Perceptron (PLA)

The perceptron computes a weighted sum of the input pixels and passes it through a step activation function:

$$\hat{y} = \text{step}(w \cdot x + b), \quad \text{step}(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Weights are updated only when a sample is misclassified, following the perceptron rule

$$w_{t+1} = w_t + \eta (y - \hat{y}) x.$$

Since a single perceptron only produces a binary output, one perceptron was trained per class in a one-vs-rest scheme (62 perceptrons in total), and the class whose perceptron produces the highest raw score $w \cdot x + b$ is chosen as the prediction. **Advantages:** simple, interpretable, cheap to train, and guaranteed to converge if the classes are linearly separable. **Limitations:** the step function has zero gradient almost everywhere so it cannot be combined with gradient-based tuning, and the weight update never converges to a low error on data that is not linearly separable, which is the case for handwritten characters.

Multilayer Perceptron (MLP)

The MLP maps the input through one or more hidden layers with a non-linear activation before the output layer:

$$h = \phi(W_1 x + b_1), \quad \hat{y} = \text{softmax}(W_2 h + b_2),$$

where ϕ is ReLU, tanh or the logistic sigmoid depending on the configuration. The network is trained by backpropagating the gradient of the multi-class cross-entropy loss

$$L = - \sum_{c=1}^{62} y_c \log(\hat{y}_c)$$

through every layer, using an optimizer (SGD or Adam) to update all weights. **Advantages:** the hidden layer lets the network learn non-linear decision boundaries, so visually similar characters can still be separated. **Limitations:** many more hyperparameters to choose (depth, width, activation, optimizer, learning rate), a much higher risk of overfitting on a small dataset, and no formal convergence guarantee.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8
Other Libraries	NumPy, Pandas, Matplotlib, Pillow
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i5, 8 GB RAM
Random State	42

8 Hyperparameter Selection

Eight MLP configurations were trained for 50 epochs each and compared on test accuracy, changing one factor at a time from the baseline (row 1):

Hidden Layers	Activation	Optimizer	LR	Batch	Test Acc.
(256,)	relu	adam	0.001	32	0.4765
(256,)	tanh	adam	0.001	32	0.4150
(256,)	logistic	adam	0.001	32	0.4692
(256,)	relu	sgd	0.010	32	0.4736
(256,)	relu	adam	0.010	32	0.4413
(256,)	relu	adam	0.001	64	0.4707
(512, 256)	relu	adam	0.001	32	0.5484
(256, 128, 64)	relu	adam	0.001	32	0.5161

Final selected MLP configuration (highest test accuracy in the sweep):

Parameter	Value
Hidden layers	(512, 256)
Activation	ReLU
Loss function	Cross-entropy (multi-class)
Optimizer	Adam
Learning rate	0.001
Batch size	32

This configuration was retrained for up to 150 epochs (early stopping on the training loss), converging after 59 epochs, to obtain the final results reported below.

9 Results

Metric	PLA	MLP (tuned)
Accuracy	0.2111	0.5499
Precision (macro)	0.2558	0.5715
Recall (macro)	0.2111	0.5499
F1-score (macro)	0.2016	0.5498
ROC-AUC (micro)	0.8115	0.9576
ROC-AUC (macro)	0.8126	0.9577

Macro averaging was used for precision, recall and F1 because the dataset is perfectly balanced across all 62 classes. Out of 682 test images, PLA classified 144 correctly and the tuned MLP classified 375 correctly. The tuned MLP reached 100% training accuracy against 54.99% test accuracy (a 45-point gap), while a version of the same network retrained with L2 regularisation ($\alpha = 0.01$) and early stopping reduced training accuracy to 92.63% and the train-test gap to 41.9 points, at a small cost to raw test accuracy (50.73%).

10 Result Visualization

Confusion matrices

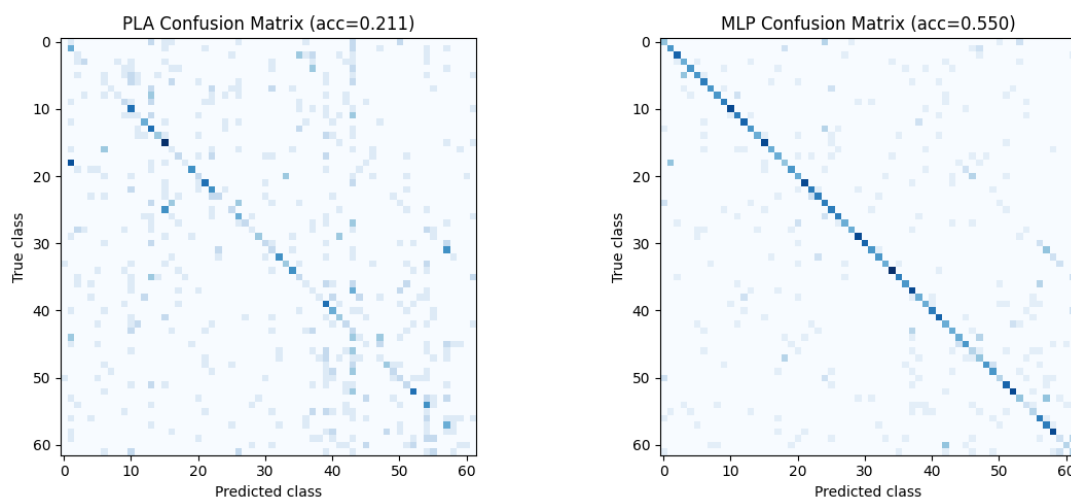


Figure 2: Confusion matrices for PLA (left) and the tuned MLP (right), 62 classes.

Inference: The MLP’s matrix shows a visibly brighter diagonal than the PLA’s, consistent with its correct count of 375 against PLA’s 144. PLA’s matrix has faint vertical banding, which means several of its 62 perceptrons fire for the same input regardless of the true class, a direct symptom of trying to separate 62 overlapping regions with straight lines. The MLP’s off-diagonal errors are more scattered and less structured, suggesting occasional confusion between visually similar characters rather than a systematic bias toward a handful of classes.

ROC curves

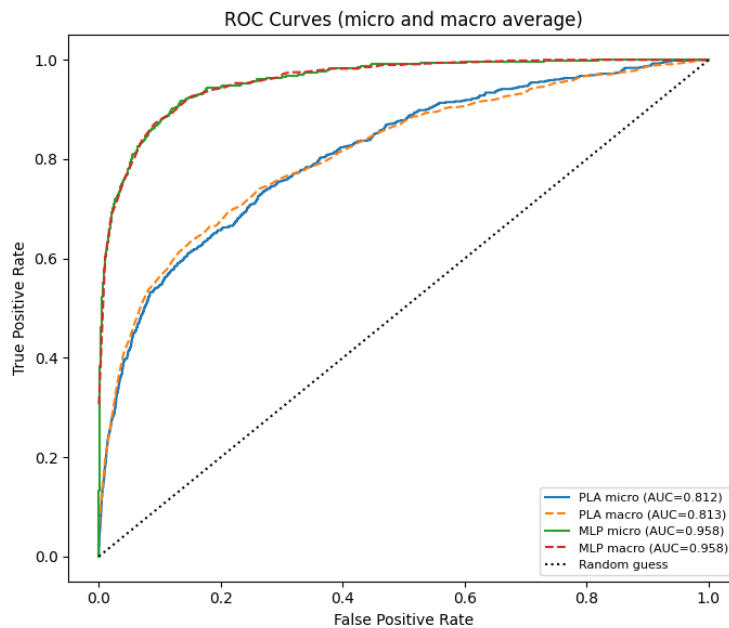


Figure 3: Micro- and macro-averaged ROC curves for PLA and the tuned MLP.

Inference: Both curves sit well above the diagonal random-guess line, but the MLP's curves (AUC 0.958 micro and macro) hug the top-left corner far more tightly than PLA's (AUC 0.812 for both). Because the micro and macro curves for each model are nearly identical, per-class performance is fairly uniform within each model, with no single class dragging the average up or down. This confirms that the 30-point accuracy gap between the two models is not an artefact of a few classes; the MLP is more confident and correct across almost the entire class set.

PLA training error

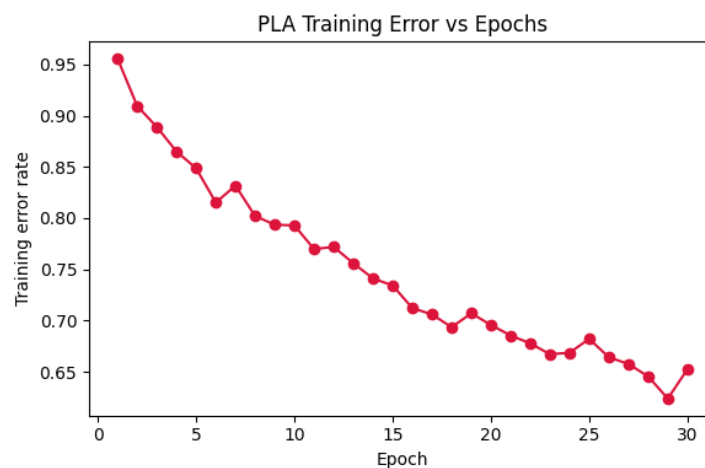


Figure 4: Fraction of training samples that triggered a weight update, per epoch.

Inference: The error rate starts at 0.956 and falls to only 0.653 after 30 epochs, a very shallow descent that has clearly not reached zero and shows no sign of doing so. This is the expected behaviour of the perceptron update rule on data that is not linearly separable: some samples will keep triggering updates indefinitely because no single hyperplane per class can separate it from all others.

MLP training loss

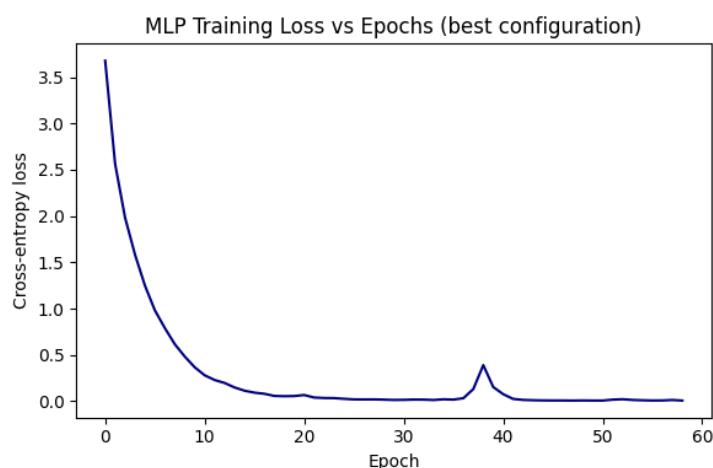


Figure 5: Cross-entropy training loss of the final MLP configuration across 59 epochs.

Inference: The loss drops sharply in the first 10–15 epochs and then continues falling more slowly to 0.0072 by epoch 59. A loss this close to zero, combined with a 100 % training accuracy, is a clear numerical signature of overfitting, which is directly reflected in the 45-point gap to the test accuracy reported in Section 9.

Optimizer comparison: SGD vs Adam

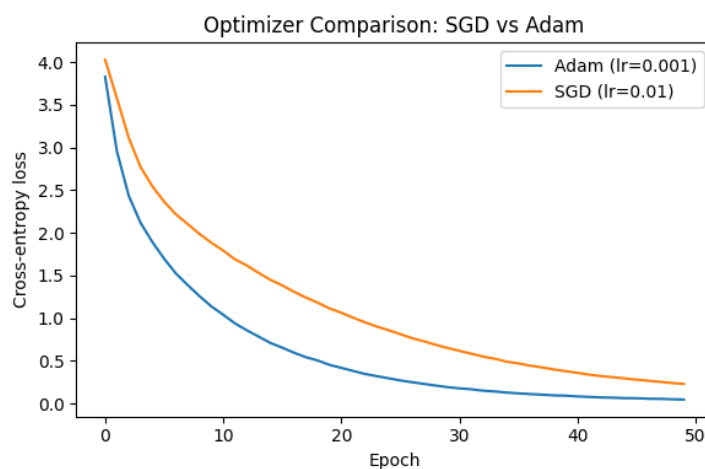


Figure 6: Training loss for the same network trained with Adam (lr 0.001) and SGD (lr 0.01), 50 epochs.

Inference: Adam’s loss curve drops faster and reaches a lower value at every epoch shown, ending near 0.05 against SGD’s 0.25 at epoch 50. Despite this clear difference in convergence speed, the final test accuracies of the two configurations were close (0.4765 for Adam, 0.4736 for SGD), showing that Adam’s adaptive step sizes mainly bought faster training in this setting rather than a noticeably better final model.

PLA vs MLP metric comparison

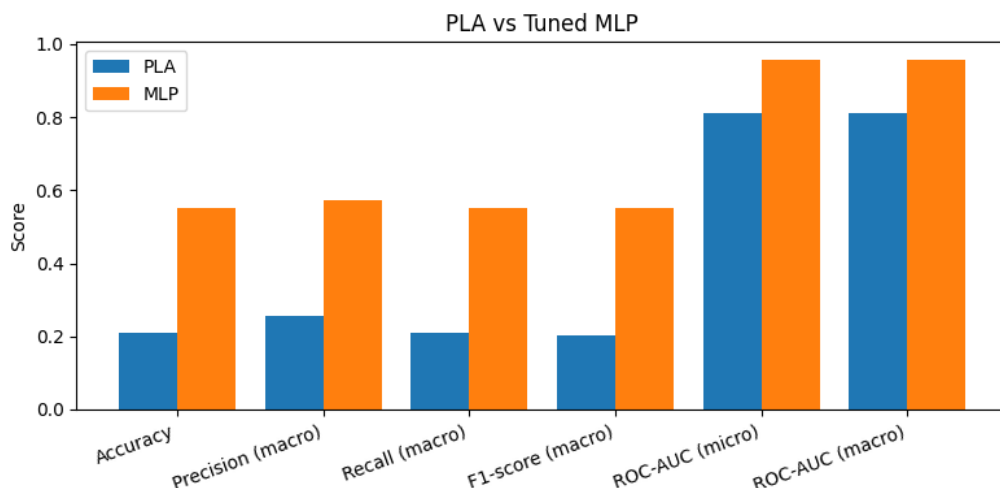


Figure 7: Side-by-side comparison of all evaluation metrics for PLA and the tuned MLP.

Inference: The MLP’s bar is taller than PLA’s on every single metric, with the widest gaps on accuracy, recall and F1-score (all roughly 0.33–0.34 higher) and the narrowest gap on ROC-AUC (about 0.15), since ROC-AUC also rewards a model for ranking the correct class highly even when its top-1 prediction is wrong. This consistent gap across every metric, not just one, is strong evidence that the MLP’s advantage is genuine and not an artefact of a single measurement choice.

11 Discussion

Why PLA underperforms. Its training error plateaus at 0.653 and never reaches zero (Section 10.3), because handwritten characters are not linearly separable and the step activation gives no usable gradient for further correction. With 62 shared linear boundaries, visually similar pairs of characters are frequently pushed to the same side.

Most impactful hyperparameters. Network capacity dominated: moving from a single 256-unit layer to (512, 256) added 7.19 points of test accuracy, the single largest gain in the sweep. Activation choice was the second largest factor, with tanh costing 6.15 points relative to ReLU. Learning rate had a smaller but still visible effect (−3.52 points at 0.01 vs 0.001), and batch size (32 vs 64) changed accuracy by less than one point.

SGD vs Adam. Adam converged noticeably faster, reaching a lower loss at every epoch in the 50-epoch comparison, but the two optimizers landed on similar final test accuracy (0.4765 vs 0.4736). Adam’s benefit here was training speed, not final quality.

Effect of adding hidden layers. Two hidden layers, (512, 256), outperformed a single layer, but adding a third layer, (256, 128, 64), reduced test accuracy from 0.5484

to 0.5161. With only 2,728 training images, the deeper and narrower network has more parameters to fit and less data per parameter, so accuracy fell rather than improved.

Overfitting. The tuned MLP reached 100 % training accuracy and a training loss of 0.0072 while test accuracy stayed at 54.99 %, a 45-point gap that is a clear overfitting signature. Retraining the same architecture with L2 regularisation ($\alpha = 0.01$) and early stopping narrowed the gap to 41.9 points (92.63 % train vs 50.73 % test), confirming that regularisation helps, though a small drop in raw test accuracy was traded for it. A larger dataset, dropout, or weight decay tuned over a wider grid would likely narrow the gap further.

12 Conclusion

A one-vs-rest perceptron and a tuned multilayer perceptron were compared on 62-class handwritten character recognition. The perceptron's linear, gradient-free updates could not separate the classes well, reaching only 21.11 % test accuracy with training error still above 65 % after 30 epochs. Systematic tuning of the MLP over activation function, optimizer, learning rate, batch size and depth identified a (512, 256) ReLU network trained with Adam as the best configuration, reaching 54.99 % test accuracy and a macro ROC-AUC of 0.958, more than double the perceptron's performance on every metric measured. The experiment also surfaced two lessons beyond the direct PLA vs MLP comparison: pixel inversion mattered more to final accuracy than any hyperparameter tested, and the tuned MLP's perfect training accuracy alongside a 45-point train-test gap is a textbook case of overfitting that partial regularisation was able to reduce but not eliminate.

13 References

<https://github.com/sugith-2025/MachineLearning/tree/main/Experiment-9>